CDC Shop – architecture discussion

Conversation between Michaela Tóthová and Claude · 29 October 2026

MICHAELA

ROLE: You are an experienced software architect
OBJECTIVE: Help me create, understand and compare 3 different architectures for the system described in CDC.txt.
SCENARIO: This is a university exercise. I will lead the discussion with follow-up questions so i understand the architectures you .
EXPECTED SOLUTION: For each architecture, give its main components, trade-offs and painful implementation parts. Write notes during our discission, it doesnt have to be 1:1 but make sure it contains everything important.
STEPS:
1. Read CDC.txt and create the notes file.
2. Sumarize what the system does and its key requirements.
3. Briefly propose 3 different architectures.
4. Then wait for my questions about the architectures.
5. Update the notes after each step.

Attachment: CDC.txt

CLAUDE

What the system does. CDC Shop is an online hardware store with two roles. Customers register, browse, search and sort products, manage a cart, pay, and track their orders. Merchants manage the catalog: they add and remove products and change prices. Both roles share login, profile, product browsing and an About Us page. The important requirements are role-based security, consistency during checkout (no paid-but-lost or unpaid-but-created orders), a read-heavy catalog that benefits from caching and search, integration with external payment, shipping and email services, and the CDC's explicit statement that the system is *distributed*. The CDC also leaves some gaps:

- whether there is one merchant or many,
- how merchants get accounts,
- whether stock is tracked,
- who updates the shipment status,
- why there is no "remove single item from cart" story.

These are good points to raise in class.

Three architectures:

- **A) Layered monolith (3-tier).** One backend with internal modules and one relational database. It is simple, and checkout is easy because everything runs in a single ACID transaction. However, it scales as one unit, coupling grows over time, and it only loosely fits the "distributed" label.

- **B) Microservices with synchronous REST.** Separate Auth, Catalog, Cart, Order, Payment, Shipping and Notification services sit behind an API gateway, each with its own database. Each service can be scaled and deployed independently. The cost is heavy operations work and fragile call chains. The hardest part is a checkout that spans several services without a shared transaction.
- **C) Event-driven (services plus a message broker).** The services are the same as in B, but they mostly talk through events such as `OrderPlaced` and `PaymentSucceeded`, and checkout is modelled as a saga. The services are loosely coupled and resilient, but the system is only eventually consistent. The painful parts are compensating actions, idempotent event handling, and debugging a single order across many services.

The details for each, including components, trade-offs and the hardest implementation parts, are in `notes.md`. What would you like to dig into first?

MICHAELA

For the architecture A i have a question. What exactly does one backend with internal modules really looks like. Is it just one project with folders like cart, orders, products?
And also if payment goes through an external service like Stripe, how can checkout be one ACID transaction? What happens if Stripe charges the customer and then my server crashes?

CLAUDE

Is it just one project with folders? Mostly yes, but the folders alone don't make modules. The difference between a modular monolith and a "big ball of mud" is a few rules. First, code is grouped by feature (`cart/`, `orders/`, `catalog/`), not by layer (`controllers/`, `services/`). Second, each module exposes a small public API, such as `CatalogService.getProduct(id)`, and keeps everything else internal. Third, each module owns its own tables, so the Order module asks Catalog for a price instead of querying the `products` table directly. Finally, dependencies go only one way, with no cycles.

You can enforce these rules with Maven/Gradle sub-modules or architecture tests (ArchUnit, Spring Modulith). Without enforcement, the boundaries erode over time. The nice side effect is that a well-kept module can later be pulled out into a microservice.

Stripe and ACID. You caught a real overstatement on my part, and I've corrected it in the notes. Checkout can't be one ACID transaction, because Stripe is outside your database and a charge can't be rolled back. This is the classic dual-write problem, and it exists even in a monolith. The usual solution splits checkout into steps:

1. In a local transaction, create the order as `PENDING_PAYMENT`, snapshot the prices and reserve stock, then commit.
2. Call Stripe outside any transaction, using the order id as the idempotency key so a retry never charges twice.

3. Treat Stripe's webhook as the source of truth. A second local transaction marks the order `PAID` or `FAILED`.

If Stripe charges the customer and your server crashes, nothing is lost. The pending order was saved before the charge, and Stripe keeps retrying the webhook until your server acknowledges it. As a safety net, a reconciliation job checks stale pending orders against Stripe's API. Two details matter here: the webhook handler must be idempotent, because Stripe may deliver the same event twice, and the browser's "success" redirect must never count as proof of payment.

So even architecture A contains a small distributed workflow at checkout, a state machine made safe by idempotency and retries. Architectures B and C face the same problem, just spread across more services. The notes now include a Q1 section with the module structure and this checkout flow.

MICHAELA

The CDC.txt says the system should be distributed. If everything is in one backend, is the architecture A you provided even distributed? Or could we just run more copies of it on different servers?

CLAUDE

Technically yes, but only weakly. A distributed system is a set of components on separate networked machines that coordinate by messages. In A, the browser, application server, database and external services like Stripe do run on different machines. All the business logic, though, lives in one deployable unit. So A is distributed in how it's deployed, not in how it's designed. Your teacher may well expect more than that, so it's worth asking what they mean.

Running more copies behind a load balancer is a legitimate way to make A properly distributed. It also gives you availability, since if one instance dies the others keep serving. The catch is that the application must become stateless, and that's where the real work is:

- **Sessions:** login state has to live in JWT tokens or Redis, not in server memory.
- **Carts:** these must also be stored in the database or Redis.
- **Background jobs:** a job like the reconciliation job must run once, not once per instance, which needs a distributed lock.
- **Webhooks:** a Stripe webhook retry can hit two instances at the same time, which makes idempotency even more important.
- **Caches:** local caches go stale on the other instances when a merchant changes a price.
- **The database:** it's still one shared component, so it becomes the bottleneck and single point of failure. Read replicas help with the read-heavy catalog but bring replication lag.

A useful way to present this is the AKF scale cube. The X-axis means cloning identical copies, which is scaled-out A. The Y-axis means splitting by business function, which is what B and C do. The Z-axis means partitioning data, for example sharding. So A becomes distributed through replication, while B and C become distributed through decomposition. That distinction should hold up well if your teacher pushes on it.

One terminology trap: don't call scaled-out A a "distributed monolith." That term refers to an anti-pattern where microservices are so tightly coupled they have to be deployed together. Call it a "horizontally scaled" or "replicated" monolith instead. All of this is now in Q2 in `notes.md`.

MICHAELA

Now i have question for architecture B. You said each service has its own database. But the Cart needs product prices and the Order needs the cart, so how does one service get data from another if it can't look into its database?

CLAUDE

"Database per service" means a service never reads another service's tables. It doesn't mean services can't share data. They share it through the owner's API, and the owner stays the single source of truth. In practice there are four ways to get the data:

1. **Ask the owner directly.** For example, Cart calls `GET /products/{id}` on Catalog. This is the default in B.
2. **Pass the data along in the request.** For example, the checkout request to Order includes the cart contents.
3. **Keep a local read-only copy.** For example, Cart stores `productId → price`, kept up to date by `PriceChanged` events. This is fast and keeps working when Catalog is down, but the copy can be slightly stale. It's also the bridge towards architecture C.
4. **Combine data from several services in one place.** The gateway or a backend-for-frontend merges the results for a single screen.

The important part is that "the price" has a different meaning in each service. Catalog owns the current price. Cart keeps a snapshot for display. Order stores the price the customer actually agreed to pay, so an order from last month doesn't change when the merchant edits the price today. That duplication is deliberate, not a design flaw.

So in B, checkout works like this:

1. Order fetches the cart items from Cart.
2. Order re-checks the current prices with Catalog in one batch call. If a price changed, the customer sees it before paying.
3. Order saves the order as `PENDING_PAYMENT`.
4. Order calls Payment, using the same Stripe pattern as in Q1.
5. Only after payment succeeds does Order ask Cart to clear itself.

The painful part is that checkout now depends on four services being up at once. Each one's availability multiplies: four services at 99.9 % give about 99.6 % together. Latency also adds up with every hop. So you need timeouts, retries only on idempotent calls, circuit breakers, and batch endpoints instead of one call per product. Where that tight runtime coupling becomes too costly, you replace some calls with event-based copies, which leads naturally into architecture C.

All of this is now in Q3 in `notes.md`.

MICHAELA

What happens if one of the services, like Catalog, is down when a customer wants to pay? Does the whole checkout just fail?

CLAUDE

In a purely synchronous B, yes, checkout fails. What matters is where and how it fails. In our flow Order calls Catalog before Payment, so if Catalog is down, the order is never confirmed and no money is taken. The customer keeps their cart and can try again. Failing safely like that is acceptable. Charging the customer based on data you couldn't verify is not.

A slow service is actually more dangerous than one that's down. A crashed service returns an error immediately, but a slow one ties up Order while it waits, and Order can then fail too. This is called a cascading failure. That's why every remote call needs a timeout. You also add a circuit breaker: after repeated failures, Order stops calling Catalog for a while and immediately shows a clear message, such as "checkout temporarily unavailable, your cart is saved."

If you want checkout to keep working while Catalog is down, there are four options:

- **Redundancy:** run several Catalog instances behind a load balancer so outages are rarer. This reduces the problem but doesn't remove it.
- **A local price copy in Order:** Order keeps its own copy of prices, updated by events. Checkout then no longer needs Catalog at runtime, at the cost of prices being possibly a few seconds out of date.
- **Honour the cart price:** accept the price saved in the cart if it's recent, say under 30 minutes old. This is a business decision rather than a technical one, because the shop accepts the risk of occasionally selling at an old price.
- **Accept the order now, validate later:** save it as pending, tell the customer "order received," and check prices and charge once Catalog is back.

It also helps to sort the services by how critical they are for checkout. Order and Payment are essential. Catalog and Cart can be replaced by a copy or by data the client sends. Shipping and Notification shouldn't be involved at all, since they can run after payment.

So the real design question in B is which dependencies stay synchronous and which become asynchronous or replicated. Once you answer that consistently, you've

essentially arrived at architecture C. This is now in Q4 in `notes.md`.

MICHAELA

For architecture C. I get that services send events instead of calling each other, but what is actually the hardest part when you really have to code it? For example, if the Payment service gets the same "OrderPlaced" event twice, could the customer be charged twice? And how do I even find out what happened to one order if it went through five services?

CLAUDE

Yes, a naive implementation would charge twice. Most brokers guarantee "at-least-once" delivery, so duplicates are normal. For example, Payment charges the card, crashes before confirming it processed the message, and the broker sends the same `OrderPlaced` again. Every consumer therefore has to be idempotent: handling an event twice must have the same effect as handling it once. For money it's usual to layer three defences:

1. **Record processed events.** Each event has a unique `eventId`. Payment saves that id in a `processed_events` table in the same database transaction as the payment, so a repeat delivery is recognised and skipped.
2. **Use a database constraint.** The `payments` table has a unique constraint on `orderId`, so one order can never have two payments.
3. **Use Stripe's idempotency key.** Using the order id as the key means even a bug in your code won't create a second charge.

In practice, though, the hardest part is the opposite problem: losing events. Order has to save the order and also publish `OrderPlaced`. If it crashes between those two steps, the order exists but no other service ever hears about it. The standard fix is the transactional outbox. Order writes the order and an "event to send" row into an `outbox` table in a single local transaction, and a separate relay process publishes those rows to the broker. The relay might send a row twice after a crash, but your idempotent consumers already handle that. Outbox on the sending side plus idempotency on the receiving side is the pairing worth remembering for the exam.

To follow one order through five services, every event and log line carries the `orderId`, or a trace id, and each service passes it on. You then collect all logs in one place (ELK or Grafana Loki) and search by that id. Distributed tracing with OpenTelemetry and Jaeger goes further: it shows the order as a single timeline across services, including where it waited or failed. A read model built from the order's events, such as `PLACED → PAID → SHIPPED`, helps with debugging and also gives you the customer's "check my order" page from story 9.

How you coordinate the services matters a lot here:

- **Choreography:** services simply react to each other's events. It's very loosely coupled, but the workflow only exists implicitly, so it's hard to see where an order got stuck.
- **Orchestration:** a saga orchestrator explicitly stores the state, such as "waiting for payment," and triggers the compensations. That gives you one place to look.

For checkout, where money and rollbacks are involved, orchestration is usually the better choice.

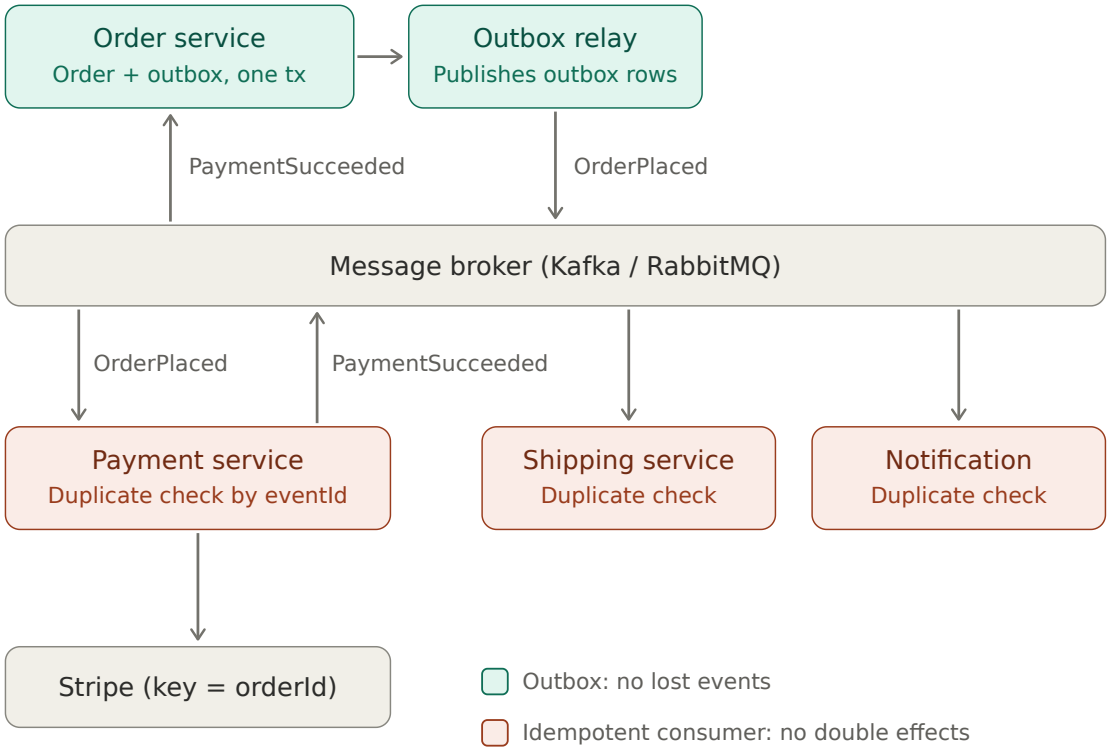
To compare with B: B struggles with services being down, while C struggles with messages being duplicated, lost, delayed or arriving out of order. The other remaining pains are ordering (Kafka keeps order only within a partition), dead-letter queues for messages that always fail, and keeping event formats compatible across versions. All of this is now in Q5 in `notes.md`.

MICHAELA

Could you show me the checkout flow in C as a simple diagram? Just the services and the events going between them, and where the outbox and the duplicate check happen. I think I'd understand it better if I see it.

CLAUDE

Here's the checkout flow in C, with the outbox on the sending side and duplicate checks on the receiving side:



Checkout flow in architecture C: outbox and duplicate checks

The diagram splits into two sides. On the teal side, the order and its outbox row are saved together in one local transaction. That's why the `OrderPlaced` event can't be lost, even if the relay crashes and has to publish it again.

On the coral side are the consumers. Each one first checks whether it has already processed that `eventId`. The Payment service also sends the `orderId` to Stripe as an idempotency key. So a duplicate event can't cause a double charge, a double shipment or a double email.

The Payment service doesn't call the Order service or Shipping directly. It just announces `PaymentSucceeded`, and each interested service reacts on its own. The failure path has exactly the same shape, with `PaymentFailed` leading to the order being cancelled and the stock released.

I've also added a text version of the diagram and the steps as Q6 in `notes.md`, so it works in your notes without the picture.

MICHAELA

Now can you compare all three? Could you make a simple table of A, B and C with things like how hard it is to build, what happens when something fails, and if it fits distributed? And if you had to pick one for CDC, which one would it be and why?

CLAUDE

	A – Modular monolith	B – Microservices (REST)	C – Event-driven
How hard to build	Low	Medium–high	High
Operations	One app, one database	Many services, gateway, tracing	Like B, plus the broker as critical infrastructure
Consistency	Strong (ACID); only the payment step needs care	No shared transaction	Eventual; sagas and compensations
When something fails	The whole app goes down unless you run copies	Failures stay in one service, but synchronous chains fail together	Services can be briefly down because events wait in the broker; new risks are duplicate, lost or reordered events
Scaling	The whole app scales as one unit	Each service scales on its own	Each service scales on its own, and the broker absorbs load spikes
Fits "distributed"	Only weakly, through replication	Yes	Yes, the most
Testing and debugging	Easy	Harder	Hardest
Checkout pain point	Coordinating with Stripe safely	Every service in the call chain must be up	Outbox, idempotency, compensations

For CDC I'd pick B, but with only a few services and events used where they clearly help, so effectively a small B+C hybrid. I'd use four or five services:

- Auth
- Catalog
- Order, with the cart inside it, because cart and order data change together
- Payment
- Shipping and Notification

Most of the stories are simple reads and CRUD, so plain REST fits them naturally. Events are used in only two places. The first is after payment, where `PaymentSucceeded` goes to Shipping and Notification. The second is a `PriceChanged` event that keeps a local price copy in Order, so checkout no longer depends on Catalog being up. Those few events get the outbox and idempotent consumers, and payment follows the Stripe pattern from Q1.

I wouldn't pick pure A because the CDC explicitly asks for a distributed system, and A meets that only weakly. I wouldn't pick pure C because the shop is too small to justify its hardest problems.

If your teacher accepts replication as "distributed", a horizontally scaled modular monolith with a background worker is the simplest correct answer. It's worth having that ready as an alternative if someone asks.

The table and the reasoning are now in section 4 of `notes.md`.

MICHAELA

You compared three architectures and then picked a mix of two. Isn't it harder to build than pure B, because we need REST and also a broker with all the outbox stuff? I would prefere one of the architectures, not a mix

CLAUDE

You're right. The hybrid keeps everything from B and adds a broker, outboxes and idempotent event consumers on top, so it's more to build, not less. It also wasn't one of the three options we compared, so it muddies the comparison. My pick for CDC is pure B.

B fits CDC because it clearly meets the "distributed" requirement. Most of the user stories are simple request–response actions, and there's only one way services talk to each other, which keeps building, testing and explaining it simpler. I'd keep the services few:

- Auth
- Catalog
- Order, which includes the cart
- Payment
- Shipping/Notification

B's weak points can be handled without a broker:

- **Catalog down at checkout:** run at least two Catalog instances, add timeouts and a circuit breaker, and fail before payment so the customer keeps their cart.
- **Double charges:** use the Stripe idempotency key, plus a unique `orderId` on payments.
- **Shipping and Notification:** Order calls them only after the order is `PAID`. If a call fails, a small retry job tries again later. The checkout has already succeeded, so this is background work, not a failed order.

If your teacher accepts replication as "distributed", a scaled-out modular monolith (A) remains the simplest answer to mention. The recommendation in `notes.md` now says pure B, with a short note on why I dropped the hybrid.

MICHAELA

So if I understand it right, we pick B because it's really distributed but still simpler than C, and we solve its weak points with timeouts, idempotency keys and a retry job instead of events ?

CLAUDE

Yes, that's right. I'd add two things.

First, redundancy belongs on that list. Running several instances of Catalog and the other critical services is what makes outages rare in the first place. Timeouts and circuit breakers only decide how the system behaves when an outage happens anyway.

Second, be ready to name the trade-off you're accepting. At the moment of checkout, Order, Payment and Catalog all have to be reachable. If one isn't, checkout fails, but it fails safely: the customer is not charged and keeps their cart. That's the price of avoiding C's complexity, and saying it explicitly will make your answer stronger if someone asks.

I added a one-sentence summary of the choice at the end of `notes.md`.